

Еще формах

- Форма и обработчик в одном файле
- Загрузка данных в форму
- Многостраничные формы
- Редактирование связанных записей

Форма и обработчик в одном файле

Изолирование от HTML

Все, что находится вне пары открывающего и закрывающего PHP-тегов, игнорируется интерпретатором PHP, у которого есть возможность обрабатывать файлы со **смешанным содержимым**. Это позволяет PHP-коду быть **встроенным** в документы HTML (изоляция PHP-кода).

`<p>Этот абзац будет проигнорирован PHP и отображен браузером.</p>`

`<?php echo '<p>Этот абзац выводится PHP интерпретатором.</p>';?>`

`<p>Этот абзац тоже будет проигнорирован PHP</p>`

Такое поведение интерпретатора весьма ожидаемо. Кратко напомним алгоритм обработки PHP файла:

- Браузер посылает серверу запрос на файл с PHP-кодом. Сервер не просто отдает клиенту (браузеру) файл, а предварительно **передает его на обработку** PHP-интерпретатору.
- PHP-интерпретатор читает содержимое файла **.php** переданного ему от сервера.
- Если интерпретатор встречает открывающий тег **<?php**, он начинает воспринимать **данные как PHP-код и выполнять** его. Результатом выполнения, как правило, является некий

сгенерированный **HTML-код** (например, являющийся результатом запроса к базе данных).

- Если интерпретатор находит закрывающий тег **?>** он перестает выполнять PHP-код.
- Все, что за пределами тегов `<?php ... ?>` (HTML-теги и текст) PHP-интерпретатор **возвращает серверу как есть**.

Так происходит за исключением случая с содержащимся внутри кода **условным оператором**, в котором интерпретатор сначала **определяет результат** условия, а уже затем **принимает решение** о выводе HTML.

Ознакомьтесь со следующим примером. Используя **альтернативный синтаксис** условной конструкции и понятие **изолирования** PHP-кода можно писать следующие инструкции:

example_1. Изолирование в PHP файлах

```
<?php $exp = true; if ($exp) : ?>

    <p>Этот абзац будет отображен, если PHP-выражение
    ИСТИННО.</p>

<?php else: ?>

    <p>Если условие ЛОЖНО, будет отображен этот абзац.</p>

    <p>... или даже несколько абзацев,</p>

    <p>сколько угодно ...</p>

<?php endif; ?>
```

В этом примере PHP **не возвращает серверу блоки, в которых условие не соблюдается**.

Даже несмотря на то, что они находятся вне пары открывающего/закрывающего тегов, PHP обработает их **в соответствии с условием**, так как интерпретатор PHP будет **игнорировать блоки**, содержащиеся внутри условия, которое не соблюдается.

Примечание. При выводе больших блоков текста **выход из режима синтаксического разбора** PHP обычно более эффективен, чем отправка текста с помощью функций **echo** или **print**.

Таким образом, при чередовании HTML и PHP кода, **оптимальнее** закрывать PHP-теги и переходить в режим HTML (выход из режима синтаксического разбора), чем использовать PHP-функции вывода строк.

Форма без атрибута action

Необязательно отправлять данные формы внешнему скрипту, данные можно обработать в том же файле, где находится сама форма.

Для того, чтобы реализовать ситуацию, когда обработчик формы и сама форма находятся в одном файле, можно использовать **проверку на существование** переменной формы с помощью PHP функции **isset()**:

```
<?php

// если страница открывается как обработчик формы

if(isset($_POST["submit"])){

    // обрабатываем данные формы

    // ...

} else {

    // если страница загружается первый раз, показываем форму

    // ...

}
```

Поскольку в данном случае мы отправляем данные этому же скрипту (то есть по этому же адресу), то у формы можно **не устанавливать атрибут action**, либо сделать его пустым.

example_2. Форма и скрипт обработчик в одном файле

```
<?php

    // если страница открывается как обработчик формы

    if(isset($_POST["submit"])):

        // заберем данные формы в переменные

        $name = $_POST["name"] ? $_POST["name"] : "No name";

        $email = $_POST["email"] ? $_POST["email"] : "не
        определен";

        // вывод в браузер

        echo "Добрый день: $name <br>";

        echo "<h2>На указанный вами e-mail ($email) отправлен
        банковский чек суммой 1.000.000 $ на предъявителя.</h2>";

        echo "Постарайтесь держать себя в рамках! ;)";

    else:

?>

    <!-- если страница загружается первый раз -->

    <h2>Введите ваши персональные данные, пожалуйста:</h2>

    <form action="" method="post">

        Имя: <input type="text" name="name"><p>

        E-mail: <input type="email" name="email"><p>

        <input name="submit" type="submit">

    </form>

<?php

    endif;

?>
```

Загрузка данных в форму

Любое приложение, как правило, выполняет все стандартные операции **CRUD**. CRUD — это аббревиатура, обозначающая четыре главные операции, используемые в приложениях для управления базами данных:

- **создание** (create),
- **чтение** (read),
- **модификация** (update),
- **удаление** (delete).

Две из операций требуют наличие форм. Использование форм для **добавления** (создания) объекта сущности мы уже практиковали, в этом уроке займемся вопросом использования форм для **редактирования данных**. Вопрос не трудный, но, тем не менее, требующий некоторых знаний.

Короткий echo-тег в PHP

PHP включает в себя короткий echo-тег **<?=**, который является сокращением для более многословного **<?php echo**.

Конструкция **<?=** означает буквально:

```
<?php echo
```

Пример использования конструкции, не вызывающий синтаксическую ошибку:

```
<!-- ОБЫЧНЫЙ ВЫВОД -->
```

```
<?php echo "<p>I'll be back</p>"; ?>
```

```
<!--
```

```
    короткий echo-тег
```

```
    иногда удобно выводить в коротком синтаксисе
```

```
-->
```

```
<?= '<h3>Пример короткого вывода</h3>'; ?>
```

Загрузка данных в текстовые поля

Редактирование данных предполагает их **загрузку в поля форм**. При этом для удобства пользователя внешний вид формы добавления и формы редактирования должен совпадать (в приложениях это чаще всего одна форма). Используя синтаксис короткого echo и изолирования PHP-кода, приведу несколько **приемов** вывода данных в **элементы формы**.

Примечание. Далее следуют довольно большие фрагменты кодов демонстрационных примеров. Ничего страшного, будьте внимательны, перед набором кода попробуйте понять алгоритм выполняемых действий.

Будем использовать известный нам прием, если **скрипт открывается первый раз**:

- **выводим список тренеров**:

```
<!-- выводим список тренеров для просмотра и редактирования -->

<p><a href='?num=0' title='Ксения Боровкова'>Ксения
Боровкова</a><p>

<!-- ... -->
```

Если скрипт открывается **после клика по ссылке**:

```
// если скрипт загружается как обработчик действия пользователя

if (isset($_GET['num'])):
```

```
// ...
```

- **подключаем** файл с массивом персональных данных тренеров;
- используя GET-параметр строки запроса, **выбираем** интересующего нас тренера;
- **загружаем** данные тренера в форму для редактирования.

example_3/index.php. Загрузка данных в текстовые поля

```
<?php

// если скрипт загружается как обработчик действия
пользователя

if (isset($_GET['num'])):

    // подключаем массив данных о тренерах

    require "coach.php";

    // из двумерного массива по индексу заберем ассоциативный
    массив интересующего нас тренера

    $coach = $coach[$_GET['num']];

?>

<a href="index.php">Назад</a><p><hr>

<form action="" method="post">

    Фамилия: <input type="text" name="surname" value="<?=
    $coach['surname']; ?>"><p>

    Имя: <input type="text" name="name" value="<?=
    $coach['name']; ?>"><p>

    E-mail: <input type="text" name="email" value="<?=
    $coach['email']; ?>"><p>

    <input name="submit" type="submit">

</form>

<?php else: ?>

<!-- выводим список тренеров для просмотра и редактирования -
```

```

->

<p><a href='?num=0' title='Ксения Боровкова'>Ксения
Боровкова</a></p>

<p><a href='?num=1' title='Владимир Комалов'>Владимир
Комалов</a></p>

<p><a href='?num=2' title='Станислав Остапенков'>Станислав
Остапенков</a></p>

<p><a href='?num=3' title='Мартина Родригес'>Мартина
Родригес</a></p>

<p><a href='?num=4' title='Виталий Ильин'>Виталий
Ильин</a></p>

<?php endif; ?>

```

example_3/coach.php. Массив данных

```

<?php

$coach = array (

    array(

        'surname' => 'Боровкова', // фамилия

        'name' => 'Ксения', // имя

        'email' => 'borovkova-k@ya.ru', // email

        'specialization' => 'Mind&Body, Stretching, Кроссфит',
        // тренерская специализация

        'post' => 'Детский фитнес, Персональный тренер', //
        основная должность

        'experience' => '5-10' // стаж работы

    ),

    // ...

```

Загрузка данных в текстовые поля – самый простой вариант использования форм редактирования. Здесь проблем быть не должно. Дальше немного сложнее.

Загрузка данных в элемент `type="radio"`

При схожем алгоритме получения персональных данных тренера есть одно существенное отличие. Необходим вывод **нескольких полей** с типом **radio** для предоставления возможности **одиночного выбора**.

Для построения формы создадим **массив значений**, из которых будем получать элементы `type="radio"`.

```
// создаем массив стажа работы
$experience = [
    '1-5',
    '5-10',
    '10-15',
    '15-20',
    '20+'
];
```

Проходим массив циклом, получаем строку элементов для добавления в форму. При этом, если текущее значение массива и значение соответствующего ключа ассоциативного массива тренера совпадают, отметим этот элемент как выбранный атрибутом – **checked**.

```
<input type="radio" name="experience" value="5-10" checked>
```

Наберите код листинга, проверьте работоспособность. Измените данные стажа тренера, проверьте отработку **вывода** скрипта.

Примечание. Естественно, запись стажа тренера должна быть **полностью идентична** одному из значений массива стажа работы. В

реальных приложениях стаж тренера берется из массива стажа, который в свою очередь, хранится где-нибудь в базе данных.

example_4/index.php. Загрузка данных в радиокнопки

```
<?php

// если скрипт загружается как обработчик действия
пользователя

if (isset($_GET['num'])):

    // подключаем массив данных о тренерах

    require "coach.php";

    // из двумерного массива по индексу заберем ассоциативный
    массив интересующего нас тренера

    $coach = $coach[$_GET['num']];

?>

<a href="index.php">Назад</a><p><hr>

<form action="" method="post">

    Фамилия: <input type="text" name="surname" value="<?=
    $coach['surname']; ?>"><p>

    Имя: <input type="text" name="name" value="<?=
    $coach['name']; ?>"><p>

    E-mail: <input type="text" name="email" value="<?=
    $coach['email']; ?>"><p>

<?php

    // создаем массив стажа работы

    $experience = [

        '1-5',

        '5-10',

        '10-15',
```

```

        '15-20',
        '20+'

    ];

    // выводим массив стажа в виде элементов
    type='radio'

    // при этом, если значение массива и значения
    соответствующего ключа тренера совпадают -
    пометим элемент как выбранный - checked

    foreach ($experience as $value) {

        if ($value == $coach['experience'])

            echo "<input type='radio'
                name='experience' value='$value'
                checked>$value<br />";

            else

                echo "<input type='radio'
                    name='experience'
                    value='$value'>$value<br />";

        }

    ?>

    <p><input name="submit" type="submit">

</form>

<?php else: ?>

    <!-- выводим список тренеров для просмотра и
    редактирования -->

    <p><a href='?num=0' title='Ксения Боровкова'>Ксения
    Боровкова</a><p>

    <p><a href='?num=1' title='Владимир Комалов'>Владимир
    Комалов</a><p>

    <p><a href='?num=2' title='Станислав
    Остапенков'>Станислав Остапенков</a><p>

```

```
<p><a href='?num=3' title='Мартина Родригес'>Мартина
Родригес</a></p>

<p><a href='?num=4' title='Виталий Ильин'>Виталий
Ильин</a></p>

<?php

    endif;

?>
```

Загрузка данных в элемент type="checkbox"

В отличие от предыдущего элемента, элементы флажков предоставляют возможность **множественного выбора**, а значит и возможность хранения **более одного** значения.

Для проверки существования элемента массива в строке хранения данных о специализации тренера используем стандартную строковую PHP-функцию.

```
// проверяем, есть ли в строке специализации тренера элемент
массива
```

```
$pos = strpos($coach['specialization'], $value);
```

Наберите код листинга, проверьте работоспособность. Измените данные специализации тренера, проверьте отработку **вывода** скрипта.

Примечание. Запись специализации тренера должна быть **полностью идентична** одному или нескольким значениям массива специализации.

example_5/index.php. Загрузка данных в чекбоксы

```
<?php
```

```

// если скрипт загружается как обработчик действия
пользователя

if (isset($_GET['num'])):

    // подключаем массив данных о тренерах

    require "coach.php";

    // из двумерного массива по индексу заберем ассоциативный
    массив интересующего нас тренера

    $coach = $coach[$_GET['num']];

?>

<a href="index.php">Назад</a><p><hr>

<form action="" method="post">

    Фамилия: <input type="text" name="surname" value="<?=
    $coach['surname']; ?>"><p>

    Имя: <input type="text" name="name" value="<?=
    $coach['name']; ?>"><p>

    E-mail: <input type="text" name="email" value="<?=
    $coach['email']; ?>"><p>

<?php

    // создаем массив тренерской специализации

    $specialization = [

        'Воркаут',

        'Кроссфит',

        'Функциональный тренинг',

        'Силовой тренинг',

        'Mind&Body',

        'Stretching',

        'Pilates Matwork',

        'Пауэрлифтинг'
    ]

```

```

];

// выводим массив специализации в виде элементов
type='checkbox'

// при этом, если значение массива найдено в
строке специализации тренера - пометим элемент
как выбранный - checked

foreach ($specialization as $value) {

    // проверяем, есть ли в строке специализации
тренера элемент массива

    $pos = strpos($coach['specialization'],
    $value);

    if ($pos === false)

        echo "<input type='checkbox'
        name='specialization[]'
        value='$value'$value<br />";

    else

        echo "<input type='checkbox'
        name='specialization[]' value='$value'
        checked >$value<br />";

}

?>

<p><input name="submit" type="submit">

</form>

<?php else: ?>

<!-- выводим список тренеров для просмотра и
редактирования -->

<p><a href='?num=0' title='Ксения Боровкова'>Ксения
Боровкова</a><p>

<p><a href='?num=1' title='Владимир Комалов'>Владимир
Комалов</a><p>

```

```
<p><a href='?num=2' title='Станислав  
Остапенков'>Станислав Остапенков</a><p>  
  
<p><a href='?num=3' title='Мартина Родригес'>Мартина  
Родригес</a><p>  
  
<p><a href='?num=4' title='Виталий Ильин'>Виталий  
Ильин</a><p>  
  
<?php endif; ?>
```

Загрузка данных в элемент список

Загрузка данных в элемент списка включает в себя использование предыдущих приемов загрузки данных в элементы формы.

Небольшое отличие в коде – условная конструкция в цикле заменена конструкцией **тернарного оператора**. Если так вам нравится больше – пожалуйста.

```
<?php  
  
    // ...  
  
    // избавились от условной конструкции в пользу тернарного  
    оператора  
  
    $option .= sprintf("<option value='%s' %s>%s</option>",  
        $value,  
        $pos === false ? "" : "selected",  
        $value  
    );  
  
    // ...
```

Обратите внимание, для элементов списка выбранный элемент отмечается атрибутом **selected**.

Пример загрузки данных продемонстрирован в коде **example_6.php**. Код опять достаточно большой, будьте аккуратны при его наборе и анализе.

example_6/index.php. Загрузка данных в элемент список

```
<?php

// если скрипт загружается как обработчик действия
пользователя

if (isset($_GET['num'])):

    // подключаем массив данных о тренерах

    require "coach.php";

    // из двумерного массива по индексу заберем ассоциативный
    массив интересующего нас тренера

    $coach = $coach[$_GET['num']];

?>

<a href="index.php">Назад</a><p><hr>

<form action="" method="post">

    Фамилия: <input type="text" name="surname" value="<?=
    $coach['surname']; ?>"><p>

    Имя: <input type="text" name="name" value="<?=
    $coach['name']; ?>"><p>

    E-mail: <input type="text" name="email" value="<?=
    $coach['email']; ?>"><p>

<?php

    // создаем массив тренерских должностей

    $post = [

        'Тренер тренажерного зала',

        'Персональный тренер',

        'Тренер групповых программ',
```

```

        'Тренер бассейна',
        'Детский фитнес',
        'Тренер по аквааэробике',
        'Тренер по йоге'

];

// выводим массив должностей в виде элементов
списка

// при этом, если значение массива найдено в
строке должности тренера - пометим элемент как
выбранный - selected

$option = "";

foreach ($post as $value) {

    // проверяем, есть ли в строке должности
тренера элемент массива

    $pos = strpos($coach['post'], $value);

    // избавились от условной конструкции в
пользу тернарного оператора

    $option .= sprintf("<option value='%s'
%s>%s</option>",

        $value,

        $pos === false ? "" : "selected",

        $value

    );

};

// обернем элементы option тегом списка select

// при этом не забудьте атрибут - multiple

echo '<select name="post[]" size=5 multiple>' .
$option . '</select><p>';

```

?>

```
<p><input name="submit" type="submit">

</form>

<?php else: ?>

    <!-- выводим список тренеров для просмотра и
    редактирования -->

    <p><a href='?num=0' title='Ксения Боровкова'>Ксения
    Боровкова</a><p>

    <p><a href='?num=1' title='Владимир Комалов'>Владимир
    Комалов</a><p>

    <p><a href='?num=2' title='Станислав
    Остапенков'>Станислав Остапенков</a><p>

    <p><a href='?num=3' title='Мартина Родригес'>Мартина
    Родригес</a><p>

    <p><a href='?num=4' title='Виталий Ильин'>Виталий
    Ильин</a><p>

<?php endif; ?>
```

Многостраничные формы

Часто возникает ситуация, когда данные для обработки серверу необходимо передать через одну или несколько **промежуточных форм**. Вспомните работу в сети Интернет, разве вы не встречали ситуации, когда сбор данных осуществлялся более чем на одной странице (ну хотя бы для того, чтобы на промежуточной форме убедить вас купить что-то очень необходимое).

Допустим, перед нами стоит задача реализовать HTML форму, которая показывает более одной страницы и **сохраняет данные** при переходе от одной страницы к следующей. Для этого можно использовать:

- **сеансы** (сессии)
- **скрытые поля** ввода.

Обратите внимание. Использование сеансов считается более безопасным способом, поскольку пользователь не может модифицировать переменные сеанса.

В качестве технологии представления передаваемых данных можно использовать:

- **сериализацию**
- **JSON-представление.**

Методом исключения получается, что для реализации многостраничных форм в этой теме нам остаются скрытые поля и JSON формат.

Сохранение в скрытых полях простых значений

Итак, начнем с более простого способа хранения и передачи данных между формами (страницами).

Создайте файлы скриптов. Протестируйте работу многостраничных форм. Эксперименты приветствуются.

example_7/index.php. Стартовая страница сбора информации

```
<form action="form2.php" method="post">
```

```
<!-- заполним данные формы и идем дальше -->
```

```
Фамилия: <input type="text" name="surname"><p>
```

```
Имя: <input type="text" name="name"><p>
```

```
Отчество: <input type="text" name="patronymic"><p>
```

```
<input type="submit">
```

```
</form>
```

Все пересылаемые данные доступны либо в GET, либо в POST массивах. В сценарии промежуточной обработки разработчику не составит труда получить данные и записать их в скрытые текстовые поля формы.

Примечание. Скрытыми полями называются элементы **input** с атрибутом **type="hidden"**.

Скрытые поля используются разработчиками не только при программировании многостраничных форм. Одно из применений рассмотрим дальше.

example_7/form2.php. Страница 2 формы

```
<form action="form3.php" method="post">

    <!-- заполним данные формы и идем дальше -->

    Уровень образования: <input type="text" name="degree"><p>

    Квалификация по диплому: <input type="text"
    name="qualification"><p>

    В каком году: <input type="text" name="date"><p>

    <!-- в скрытые поля запишем данные первой страницы -->

    <input type="hidden" name="surname" value='<?=$_POST["surname"]; ?>'><p>

    <input type="hidden" name="name" value='<?=$_POST["name"]; ?>'><p>

    <input type="hidden" name="patronymic" value='<?=$_POST["patronymic"]; ?>'><p>

    <input type="submit">

</form>
```

Конечному сценарию абсолютно все равно, сколько промежуточных форм прошли получаемые данные. Мы просто извлекаем данные массивов POST или GET.

example_7/form3.php. Страница 3 формы

```
<form action="result.php" method="post">

    <!-- заполним данные формы и идем дальше -->

    ИНН <input type="text" name="inn"><p>

    СНИЛС <input type="text" name="snils"><p>

    Полис ОМС <input type="text" name="oms"><p>

    <!-- в скрытые поля запишем данные первой страницы -->

    <input type="hidden" name="surname" value='<?=$_POST["surname"]; ?>'><p>

    <input type="hidden" name="name" value='<?=$_POST["name"]; ?>'><p>

    <input type="hidden" name="patronymic" value='<?=$_POST["patronymic"]; ?>'><p>

    <!-- в скрытые поля запишем данные второй страницы -->

    <input type="hidden" name="degree" value='<?=$_POST["degree"]; ?>'><p>

    <input type="hidden" name="qualification" value='<?=$_POST["qualification"]; ?>'><p>

    <input type="hidden" name="date" value='<?=$_POST["date"]; ?>'><p>

    <input type="submit">

</form>
```

Конечный скрипт выводит данные, как если бы мы использовали одну страницу формы сбора пользовательских данных.

example_7/result.php. Конечный вывод данных формы

```
<?php

    if(isset($_POST['surname'])) {

        echo "<pre>";

        print_r($_POST);

        echo "</pre>";

        echo "<h2>{"$_POST['surname']} {"$_POST['name']} {"$_POST['patronymic']}</h2>";

        echo "<h3><i>Уровень образования:</i> {"$_POST['degree']}</h3>";

        echo "<h3><i>Квалификация по диплому:</i> {"$_POST['qualification']}</h3>";

        echo "<h3><i>В каком году окончил:</i> {"$_POST['qualification']}</h3>";

        echo "<h3><i>ИНН:</i> {"$_POST['inn']}</h3>";

        echo "<h3><i>СНИЛС:</i> {"$_POST['snils']}</h3>";

        echo "<h3><i>Полис ОМС:</i> {"$_POST['oms']}</h3>";

    }

?>
```

Сохранение в скрытых полях JSON-представления

Если необходимо сохранить промежуточные данные небольшого объема, то проблем не возникнет (вопросы безопасности пока не рассматриваем), но если речь идет о данных большого объема или данных более сложных структур, то на помощь нам снова приходит **JSON формат**.

Модернизируем файлы скриптов примера **example_7**. Протестируем работу **многостраничных форм** с использованием данных **JSON формата**. Эксперименты снова приветствуются (всегда).

example_8/index.php. Стартовая страница сбора информации

```
<h2>Заполните общие персональные данные. Страница 1</h2>

<form action="form2.php" method="post">

    <!-- заполним данные формы и идем дальше -->

    Фамилия: <input type="text" name="data[surname]"><p>

    Имя: <input type="text" name="data[name]"><p>

    Отчество: <input type="text" name="data[patronymic]"><p>

    <input type="submit">

</form>
```

Кодируем принятые данные в JSON формат, записываем данные в скрытое поле. Важно на следующих страницах не забыть, что в текстовом поле **не строковые данные**, а **JSON представление PHP массива** с данными формы.

example_8/form2.php. Страница 2 формы

```
<h2>Заполните данные об образовании. Страница 2</h2>

<?php

    // кодируем данные первой страницы в JSON-строку

    $json = json_encode($_POST["data"]);

?>

<form action="form3.php" method="post">

    <!-- заполним данные формы и идем дальше -->

    Уровень образования: <input type="text"

    name="data[degree]"><p>

    Квалификация по диплому: <input type="text"
```

```

name="data[qualification]"><p>

В каком году: <input type="text" name="data[date]"><p>

<!-- в скрытое поле запишем JSON-представление данных первой
страницы -->

<input type="hidden" name="json" value='<?php echo $json;
?>'><p>

<input type="submit">

</form>

```

Необходимо декодировать JSON строку в PHP объект для **слияния массивов** первой и второй страниц. Далее снова закодировать в JSON и опять записать в скрытое поле.

example_8/form3.php. Страница 3 формы

```

<h2>Заполните специальные данные. Страница 3</h2>

<?php

// декодируем данные первой формы в PHP массив
$data = json_decode($_POST["json"], true);

// объединяем массивы с данными первой и второй страниц
$arr = array_merge($data, $_POST['data']);

// кодируем объединенные данные в JSON-строку
$json = json_encode($arr);

?>

<form action="result.php" method="post">

<!-- заполним данные формы и идем дальше -->

ИНН <input type="text" name="data[inn]"><p>

СНИЛС <input type="text" name="data[snils]"><p>

Полис ОМС <input type="text" name="data[oms]"><p>

<!-- в скрытое поле запишем JSON-представление объединенных

```

```
данных -->

<input type="hidden" name="json" value='<?php echo $json;
?>'><p>

<input type="submit">

</form>
```

Данные пришли в конечный скрипт обработки и вывода.

example_8/result.php. Собираем полученную информацию

```
<?php

// декодируем полученные данные двух страниц
$data = json_decode($_POST["json"], true);

// объединяем массивы с данными

$arr = array_merge($data, $_POST['data']);

echo "<pre>";

print_r($arr);

echo "</pre>";

echo "<h2>{$arr['surname']} {$arr['name']}
{$arr['patronymic']}</h2>";

echo "<h3><i>Уровень образования:</i>
{$arr['degree']}</h3>";

echo "<h3><i>Квалификация по диплому:</i>
{$arr['qualification']}</h3>";

echo "<h3><i>В каком году окончил:</i>
{$arr['qualification']}</h3>";

echo "<h3><i>ИНН:</i> {$arr['inn']}</h3>";

echo "<h3><i>СНИЛС:</i> {$arr['snils']}</h3>";

echo "<h3><i>Полис ОМС:</i> {$arr['oms']}</h3>";

?>
```

В одной из работ самостоятельных заданий рассмотрим еще один пример использования скрытых полей и формата JSON.

Редактирование связанных записей

Еще один важный вопрос. Даже если вы никогда не программировали баз данных, в темах, посвященных массивам, и, в частности, связанным массивам, я говорил про понятия **первичного** и **внешнего** ключей.

Вот выдержка из темы о связанных массивах.

К сведению. Совсем не обязательно пытаться вникнуть в основы реляционных баз данных. Достаточно согласиться, что все написанное о разделении таблиц - естественно и логично. А еще свыкнуться с мыслью, что понятие **первичного** и **внешнего** ключа будет преследовать вас все время разработки веб-приложений.

Настоятельно рекомендую освежить в памяти содержимое, если в этом есть необходимость. Если совсем кратко, то основной посыл следующий:

- **Реляционная база данных (РБД)**– это структура хранения взаимосвязанных элементов информации, организованных в виде **набора таблиц**.
- В таблицах хранится информация **о сущностях** базы данных. **Сущность** — это логическая концепция модели реляционной базы данных (примеры сущностей: Авторы книг, Книги, Студенты, Товары и т.д.). В каждой таблице хранится информация об определенной сущности.
- Между таблицами баз данных существуют предопределенные **связи** (отношения). Действительно, между таблицей, хранящей

информацию об авторах и их книгами, между таблицей "Студенты" и их успеваемостью должны быть установлены жесткие связи.

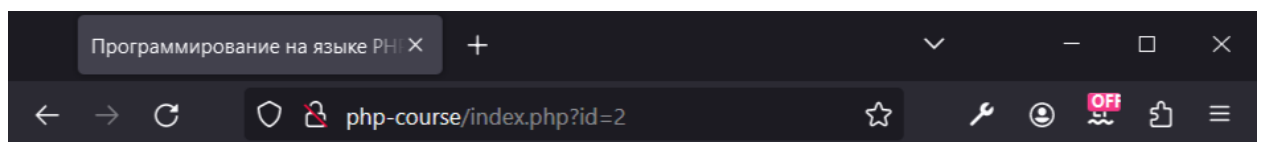
- Для **осуществления связей** (и не только) используются **первичные** и **внешние** ключи.

В этом разделе поговорим про **внешние** ключи.

Рассмотрим код следующего листинга. Не забудьте к адресу страницы добавить **GET-параметр id** (в диапазоне 1-6).

example_9/index.php. Передача параметра для связанных записей

```
<?php
```



Еще о формах

Вы запросили информацию по группе:

Pink Floyd

Идентификатор группы: 2
Страна: **Великобритания**
Год основания: **1965**
Стиль исполнения: **психоделический-рок**
Примечание:

Добавьте альбом в дискографию группы

Добавить альбом

В подключенном массиве нашли и вывели в браузер интересующую нас группу. Теперь хотим добавить к группе новый альбом, для этого в коде реализована ссылка на **форму добавления** новой записи.

Примерный вид формы выведен на рисунке.

Еще о формах

Добавить альбом

Название альбома:

Страна записи:

Лейбл звукозаписи:

Год записи:

Примечание:

Идентификатор группы:

Пользователь, заполняющий такую форму, столкнется с проблемой. Поле, определяющее принадлежность альбома к какой-либо группе (**внешний ключ**), останется пустым. Записать в **файл** или **базу данных** информацию из формы можно, но целостность такой записи будет нарушена. **Есть альбом, не принадлежащий ни одной из групп.**

Вполне возможно, что в реальной жизни такая ситуация и вероятна, но сейчас мы точно этого не хотели.

Выход достаточно прост. В коде предыдущего сценария **идентификатор группы** фигурировал в виде GET-параметра. Задача разработчика сделать так, чтобы этот параметр каким-либо образом попал в **скрипт вывода формы добавления альбома**.

К сведению. Передача **значения внешнего ключа** – тривиальная задача разработчика приложения, программирующего базы данных.

Рассмотрим, как это можно сделать.

И как обычно, нет хорошего или плохого способа, есть поставленная задача и возможные пути ее решения.

Использование JavaScript кода

Мы не изучаем JavaScript, но тем не менее, вот два способа организовать переход на страницу формы с передачей дополнительного параметра. В нашем случае этим параметром будет являться **идентификатор группы**.

- с помощью тега **button** и атрибута **onclick** (кнопки):

```
<button onclick="document.location='form.php?id=<?=$_GET['id']; ?>' ">Добавить альбом</button>
```

- с помощью тега **a** и атрибута **href** (ссылки):

```
<a href="form.php?id=<?=$_GET['id']; ?>">Добавить альбом</a>
```

Реализация передачи параметра и вывода его в поле формы приведена в файлах директории **example_10**.

example_10/index.php.

```
<?php

// не забудьте к адресу страницы добавить GET-параметр id
// например, у меня адрес выглядит следующим образом:
// http://php-course/index.php?id=2
// подключим массив групп

require "team.php";

if(isset($_GET['id'])) {

    // найдем группу с указанным в GET-параметре
    идентификатором
```

```

    foreach ($team as $item) {

        if ($_GET['id'] == $item["id_team"]) {

            // вернем группу в массив

            $team = $item;

            break;

        };

    };

    echo "<h3>Вы запросили информацию по группе:</h3>";

    echo <<<HERE

        <h2>{$team["name"]} </h2>

        Идентификатор группы: <b>{$team["id_team"]} </b><br>

        Страна: <b>{$team["country"]} </b><br>

        Год основания: <b> {$team["date"]} </b><br>

        Стил ь исполнения: <b> {$team["style"]} </b><br>

        Примечание: <b> {$team["note"]} </b><br>

        <hr>

        HERE;

    }

?>

<h2>Добавьте альбом в дискографию группы</h2>

<button onclick="document.location='form.php?id=<?=$_GET['id']; ?>' ">Добавить альбом</button><p>

<a href="form.php?id=<?=$_GET['id']; ?>">Добавить альбом</a>

```

example_10/form.php.

```

<form method="post">

```

```
Название альбома: <input type="text" name="name"><p>

Страна записи: <input type="text" name="country"><p>

Лейбл звукозаписи: <input type="text" name="lable"><p>

Год записи: <input type="text" name="date"><p>

Примечание: <input type="text" name="note"><p>

<!-- здесь должен храниться внешний ключ для связи таблиц -->

Идентификатор группы: <input type="text" name="id_team"
style="border-color: red;" value='<?= $_GET['id'];?>' ><p>

<input type="submit" value="Добавить">

</form>
```

Использование формы

А можно это сделать с использованием формы и скрытого поля:

```
<form action="form.php" method="post">

    <input type="hidden" name="id" value='<?= $_GET['id'];?>'>

    <button>Добавить альбом</button>

</form>
```

Реализация передачи параметра и вывода его в поле формы приведена в файлах директории **example_11**.

example_11/index.php.

```
<?php

// не забудьте к адресу страницы добавить GET-параметр id

// например, у меня адрес выглядит следующим образом:

// http://php-course/index.php?id=2
```

```

if(isset($_GET['id'])) {

    // подключим массив групп

    require "team.php";

    // найдем группу с указанным в GET-параметре
    идентификатором

    foreach ($team as $item) {

        if ($_GET['id'] == $item["id_team"]) {

            // вернем группу в массив

            $team = $item;

            break;

        };

    };

    echo "<h3>Вы запросили информацию по группе:</h3>";

    echo <<<HERE

        <h2>{$team["name"]} </h2>

        Идентификатор группы: <b>{$team["id_team"]} </b><br>

        Страна: <b>{$team["country"]} </b><br>

        Год основания: <b> {$team["date"]} </b><br>

        Стил ь исполнения: <b> {$team["style"]} </b><br>

        Примечание: <b> {$team["note"]} </b><br>

        <hr>

        HERE;

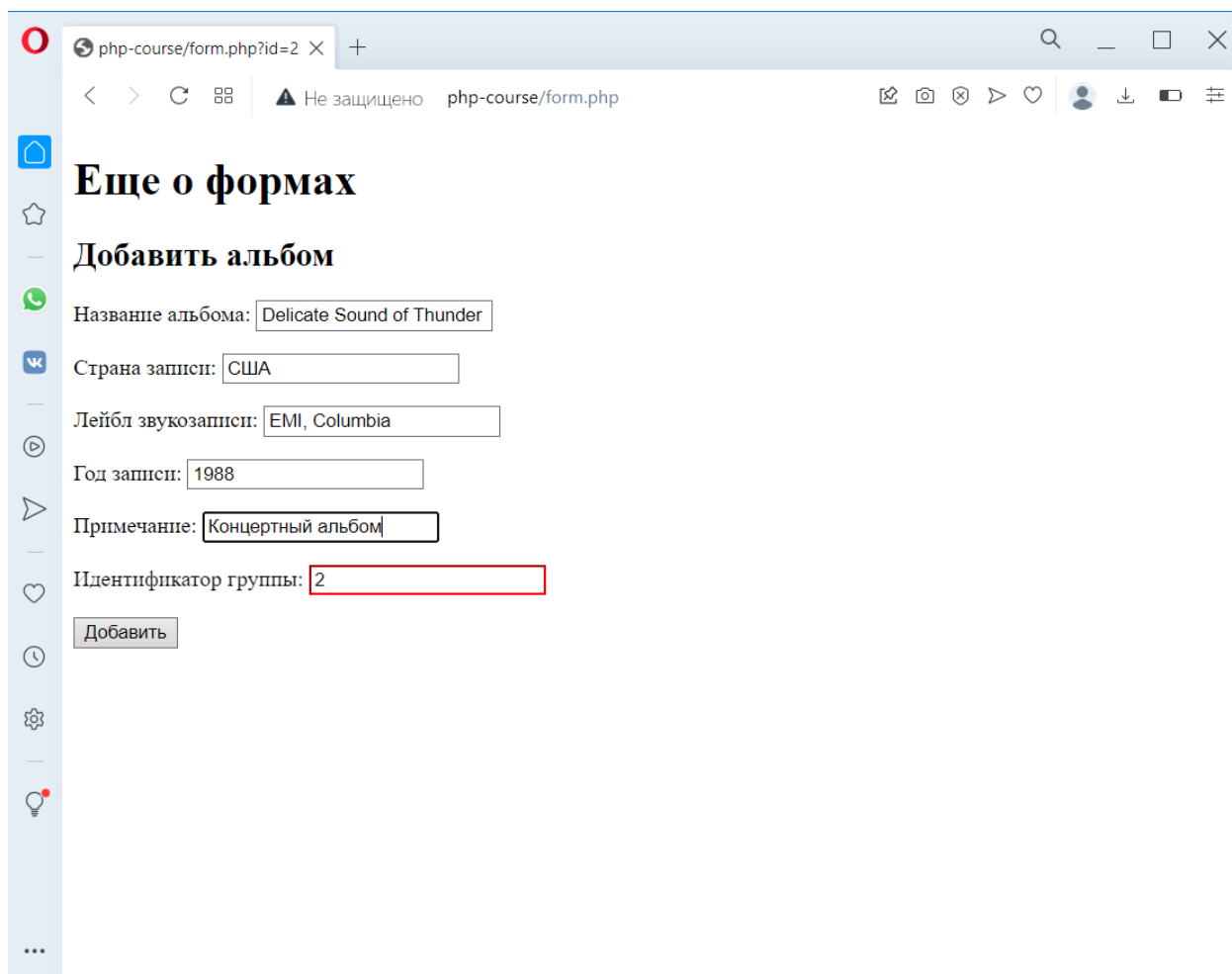
    }

?>

```

Вариантов решения несколько. Но абсолютно точно, заполнять форму, представленную на рисунке ниже гораздо приятнее.

Удачи.



The screenshot shows a web browser window with the address bar displaying 'php-course/form.php?id=2'. The page title is 'Еще о формах'. Below the title is a section 'Добавить альбом' (Add album). The form contains the following fields:

- Название альбома: Delicate Sound of Thunder
- Страна записи: США
- Лейбл звукозаписи: EMI, Columbia
- Год записи: 1988
- Примечание: Концертный альбом
- Идентификатор группы: 2

A 'Добавить' (Add) button is located below the form fields. The 'Идентификатор группы' field is highlighted with a red border.

P.S. Скрыть поле формы "Идентификатор группы" или оставить как есть – решать вам. Иногда могут быть причины оставить его видимым, но заблокировать от случайного изменения точно стоит.